```
"""
AI MODEL EVALUATION & DRIFT MONITORING  (credit-risk classifier)
----------------------------------------------------------------
Goal: build a simple ML model, then do what an auditor would do:
  1. Evaluate accuracy / reliability   (accuracy, precision, recall, ROC-AUC, cross-validation)
  2. Test robustness                   (add noise to inputs, see how much performance drops)
  3. Monitor drift year over year      (PSI + KS test on inputs and on model outputs)
  4. Design a human-in-the-loop control (low-confidence predictions go to manual review)
Run:  pip install scikit-learn pandas numpy scipy   then   python ai_model_evaluation_drift_project.py
Dataset: German Credit (OpenML "credit-g"). Falls back to synthetic data if offline.
NOTE: "Year 2" data is SIMULATED by shifting some inputs. Say this clearly in your README/interview.
"""
import numpy as np, pandas as pd
from scipy.stats import ks_2samp
from sklearn.datasets import fetch_openml, make_classification
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_auc_score

rng = np.random.default_rng(42)

# ---------- 1. LOAD DATA ----------
def load():
    try:
        d = fetch_openml("credit-g", version=1, as_frame=True)
        return d.data, (d.target == "bad").astype(int), "German Credit (OpenML)"
    except Exception:
        Xn, y = make_classification(n_samples=1000, n_features=8, n_informative=5,
                                    weights=[0.7, 0.3], random_state=42)
        X = pd.DataFrame(Xn, columns=[f"feature_{i}" for i in range(8)])
        return X, pd.Series(y), "Synthetic fallback"

X, y, source = load()
print(f"Data source: {source} | rows={len(X)} | default rate={y.mean():.1%}")
num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = [c for c in X.columns if c not in num_cols]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y,
random_state=42)

def make_pipe(model):
    pre = ColumnTransformer([("num", StandardScaler(), num_cols),
                             ("cat", OneHotEncoder(handle_unknown="ignore"),
cat_cols)])
    return Pipeline([("pre", pre), ("model", model)])

# ---------- 2. MODEL EVALUATION ----------
models = {"Logistic Regression": make_pipe(LogisticRegression(max_iter=1000)),
          "Random Forest": make_pipe(RandomForestClassifier(n_estimators=200,
random_state=42))}
rows = []
for name, pipe in models.items():
    cv_auc = cross_val_score(pipe, X_train, y_train, cv=5, scoring="roc_auc").mean()
    pipe.fit(X_train, y_train)
    p = pipe.predict_proba(X_test)[:, 1]; pred = (p >= 0.5).astype(int)
    rows.append({"model": name, "cv_auc": cv_auc, "accuracy": accuracy_score(y_test,
pred),
                 "precision": precision_score(y_test, pred), "recall":
recall_score(y_test, pred),
                 "test_auc": roc_auc_score(y_test, p)})
results = pd.DataFrame(rows).round(3)
print("\n=== MODEL EVALUATION ===\n", results.to_string(index=False))
best_name = results.sort_values("test_auc", ascending=False).iloc[0]["model"]
best = models[best_name]; print(f"\nSelected model: {best_name}")

# ---------- 3. ROBUSTNESS (noise test) ----------
print("\n=== ROBUSTNESS: add noise to numeric inputs ===")
base_auc = roc_auc_score(y_test, best.predict_proba(X_test)[:, 1])
for lvl in [0.1, 0.3, 0.5]:
    Xn = X_test.copy()
    for c in num_cols:
        Xn[c] = Xn[c] + rng.normal(0, lvl * X_train[c].std(), len(Xn))
    auc = roc_auc_score(y_test, best.predict_proba(Xn)[:, 1])
    print(f"noise={lvl:.1f} std -> AUC {auc:.3f} (change {auc-base_auc:+.3f})")

# ---------- 4. DRIFT MONITORING (Year 1 vs simulated Year 2) ----------
def psi(base, new, bins=10):
    edges = np.unique(np.quantile(base, np.linspace(0, 1, bins + 1)))
    edges[0], edges[-1] = -np.inf, np.inf
    b = np.histogram(base, edges)[0] / len(base); n = np.histogram(new, edges)[0] /
len(new)
    b, n = np.clip(b, 1e-4, None), np.clip(n, 1e-4, None)
    return float(np.sum((n - b) * np.log(n / b)))

X_y2 = X_test.copy()                      # simulate "Year 2": shift two numeric
inputs
```

```
for c in num_cols[:2]:
    X_y2[c] = X_y2[c] * 1.15 + 0.5 * X_train[c].std()

print("\n=== DRIFT: PSI (<0.1 stable, 0.1-0.25 watch, >0.25 significant) ===")
drift = []
for c in num_cols:
    s = psi(X_train[c].values, X_y2[c].values); ks = ks_2samp(X_train[c], X_y2[c])
    drift.append({"feature": c, "PSI": round(s, 3), "KS_p": round(ks.pvalue, 4),
                  "status": "SIGNIFICANT" if s > 0.25 else "WATCH" if s > 0.1 else
"stable"})
print(pd.DataFrame(drift).to_string(index=False))
p1 = best.predict_proba(X_test)[:, 1]; p2 = best.predict_proba(X_y2)[:, 1]
print(f"\nOutput (predicted probability) PSI: {psi(p1, p2):.3f}")
print(f"Mean predicted default risk: Year1 {p1.mean():.3f} -> Year2 {p2.mean():.3f}")

# ---------- 5. HUMAN-IN-THE-LOOP CONTROL ----------
p = best.predict_proba(X_test)[:, 1]
review = (p >= 0.35) & (p <= 0.65)          # low-confidence zone -> manual review
auto = ~review
acc_auto = accuracy_score(y_test[auto], (p[auto] >= 0.5).astype(int))
acc_rev = accuracy_score(y_test[review], (p[review] >= 0.5).astype(int)) if
review.any() else float("nan")
print("\n=== HUMAN-IN-THE-LOOP CONTROL ===")
print(f"Routed to manual review: {review.mean():.1%} of cases")
print(f"Accuracy auto-decided: {acc_auto:.3f} | accuracy in review zone:
{acc_rev:.3f}")
print("KPI monitoring control: alert if any input PSI > 0.25 or review rate > 40%.")
```